Cureus
Part of SPRINGER NATURE

Text Sentiment Analyser: A Hybrid Rule-Based and Machine Learning Framework for Sentiment and Emotion Analysis of English and Code-Mixed Hinglish Text

Sudhanshu Shrivastav[1], Ravi Kumar Bhardwaj[1] and Audarsh Singh Rathore[1]

[1] Department of Computer Science and Engineering, Maharana Institute of Professional Studies, Kanpur, Uttar Pradesh 209217, India (affiliated to Dr. A.P.J. Abdul Kalam Technical University, Lucknow)

Abstract. Indian social-media users often type Hindi in the Latin alphabet and switch to English within one sentence, a style known as Hinglish. English sentiment tools handle such text poorly because Romanised Hindi opinion words are missing from their lexicons, spellings vary widely and Hindi places the negator after the word it negates. This paper presents Text Sentiment Analyser, a privacy-preserving system that labels English and Hinglish text as Positive, Neutral or Negative and profiles its emotions. It combines language identification and spelling normalisation, a transparent bilingual lexicon engine with pre- and post-negation, intensifier and contrast rules, multinomial Naive Bayes and logistic regression over TF-IDF n-grams, and a probabilistic hybrid of rules and Naive Bayes, together with modules for eight-class emotion detection, aspect-level sentiment, sarcasm cues and word-level explanation. Five engines were compared by stratified five-fold cross-validation on a balanced pilot corpus of 210 texts. An English-only lexicon reached 55.2% accuracy, the Hinglish-aware engine 92.4% (95% CI 88.6–95.7), Naive Bayes 65.9% and the hybrid 82.1% over five random splits, a significant gain over Naive Bayes (McNemar $p < 0.001$). Because the lexicon was built alongside the corpus, the rule-based figure is an optimistic bound; ablation, learning-curve and error analyses show where each component helps. The whole system runs offline in a web browser.

Keywords: *Sentiment analysis, Code-mixed text, Hinglish, Natural language processing, Machine learning, Lexicon-based methods, Naive Bayes, Emotion detection, Hybrid model*

Introduction:

A. Background and Motivation

Sentiment analysis is the computational study of the opinions and feelings that people express in text. It supports product-review mining, brand monitoring, complaint triage and opinion research [1–3]. The best-known tools, including the rule-based analysers VADER [4] and SO-CAL [5] and the supervised classifiers popularised by Pang et al. [6], were built and tested almost entirely on English. In India, however, many users switch between Hindi and English within a sentence and type Hindi with a Roman keyboard, the default input method on most phones [7]. The resulting style, Hinglish, is illustrated by *"movie acchi nahi thi but songs mast the"* (the movie was not good, but the songs were great).

Such text defeats English tools for three reasons. *Vocabulary*: Hindi words such as *bakwas* (rubbish), *zabardast* (excellent) and *bekaar* (useless) carry strong polarity but are absent from English lexicons, so they are scored as neutral. *Spelling*: Romanised Hindi has no standard orthography, so *bahut* (very) also appears as *bahot*, *bhot* or *bohot*, and *accha* (good) as *acha*, *acchi*

or *achi*. *Word order*: Hindi places the negator after the word it negates (*acchi nahi thi*), so the forward-looking negation windows of English heuristics point the wrong way. Code-mixing is known to harm language identification, tagging and sentiment classification alike [8–10].

This paper describes Text Sentiment Analyser, a sentiment and emotion analysis system for English and Hinglish that combines natural language processing (language identification, normalisation, tokenisation and n-gram features), a hand-built and fully explainable rule engine, and machine learning (Naive Bayes, logistic regression and a probabilistic hybrid). All processing runs on the client, so no text leaves the user's device.

B. Problem Statement and Objectives

English-only tools ignore Romanised Hindi opinion words, apply negation to the wrong word and treat spelling variants as unknown, so many Hinglish comments are wrongly labelled Neutral. Supervised models could learn this vocabulary but need labelled Hinglish data that small projects rarely have. The objectives are therefore (i) to design a Hinglish-aware rule engine with normalisation, bilingual negation, intensifiers and contrast handling; (ii) to train and compare Naive Bayes and logistic regression on a labelled corpus; (iii) to propose a hybrid of rules and Naive Bayes and quantify when fusion helps; (iv) to add emotion, aspect, sarcasm and explanation modules; and (v) to evaluate all engines with reproducible statistics and report limitations openly.

C. Contributions

The contributions are: (1) a complete, dependency-free Hinglish sentiment and emotion pipeline with 292 English and 184 Hinglish lexicon entries, 28 Hindi intensifiers and 94 spelling-variant mappings; (2) a five-engine comparison under repeated stratified cross-validation with bootstrap confidence intervals, Cohen's kappa and McNemar tests; (3) component ablations, a hybrid-weight sweep and learning curves; and (4) an error taxonomy of the remaining failures.

Literature Review:

This section reviews the studies that inform the design of the system, concentrating on work published between 2020 and 2026 together with the foundational studies it builds on. The most relevant studies are summarised in Table 1.

A. Surveys of Sentiment Analysis

[3] Birjali et al. (2021) surveyed lexicon-based, machine-learning and hybrid sentiment methods and their common challenges, including negation, domain dependence, sarcasm and multilingual text, and observed that hybrid designs combine interpretability with adaptability. [11] Wankhade et al. (2022) reviewed methods and applications across document, sentence and aspect levels and identified code-mixed, low-resource languages as an open gap. [12] Nandwani and Verma (2021) reviewed sentiment analysis together with emotion detection and described emotion models such as Plutchik's wheel.

[13] Bordoloi and Biswas (2023) proposed a generic design framework running from data collection and pre-processing to classification and visualisation, which closely matches the pipeline used here. [14] Tan et al. (2023), [15] Rodríguez-Ibánez et al. (2023) and [16] Jim et al. (2024) reviewed approaches, datasets and social-media applications, and all identified informal, emoji-rich and code-switched text as persisting difficulties. [17] Jain et al. (2021) reviewed

machine learning for consumer reviews, and [18] Hartmann et al. (2023) compared methods across 272 marketing datasets, finding transfer learning most accurate but lexicons faster and more transparent.

B. Lexicon-Based and Hybrid Methods

[4] Hutto and Gilbert (2014) proposed VADER, a validated lexicon with heuristics for punctuation, capitals, degree words, the conjunction *but* and negation, normalised into a compound score; our compound score follows VADER, which however knows no Hindi. [5] Taboada et al. (2011) modelled intensification and negation as systematic shifts of word polarity in SO-CAL, which inspired our attenuated negation factor. [19] Cambria et al. (2022) presented SenticNet 7, a neurosymbolic framework that uses commonsense knowledge to make sentiment analysis explainable.

[20] Catelli et al. (2022) compared a lexicon analyser with BERT on Italian and found BERT more accurate but the lexicon faster, data-free and interpretable, the same trade-off studied here. [21] Appel et al. (2016) showed that combining lexicons with classifiers improves sentence-level analysis, and [22] Kaur and Sharma (2023) and [23] Alsayat (2022) proposed hybrid feature and ensemble models for reviews and social media, motivating our probabilistic fusion.

C. Machine Learning and Deep Learning

[6] Pang et al. (2002) showed that Naive Bayes, maximum entropy and support vector machines beat human-chosen keywords for review classification, and [24] Wang and Manning (2012) showed that bigram features with linear models remain strong baselines, justifying our feature set. [25] Devlin et al. (2019) introduced BERT, and [26] Tan et al. (2022) combined RoBERTa with an LSTM to outperform baselines on IMDb, Twitter airline and Sentiment140 data. [27] Barbieri et al. (2022) released XLM-T, a multilingual Twitter model for sentiment in eight languages; none of these resources targets Romanised Hinglish.

D. Code-Mixing and Hinglish Processing

[7] Bali et al. (2014) showed that English-Hindi switching on Facebook occurs frequently inside sentences, while [8] Barman et al. (2014) and [9] Vyas et al. (2014) showed that code-mixing degrades language identification and tagging, which justifies our identification and normalisation stage. [28] Doğruöz et al. (2021) surveyed code-switching from linguistic and social perspectives, and [29] Winata et al. (2023) found that Hindi-English is among the most studied pairs but that resources remain fragmented.

[30] Aguilar et al. (2020) and [31] Khanuja et al. (2020) proposed the LinCE and GLUECoS benchmarks for code-switched NLP, including Hindi-English sentiment, and Khanuja et al. reported gains from fine-tuning multilingual BERT on synthetic code-switched text. [32] Nayak and Joshi (2022) built L3Cube-HingCorpus and pre-trained HingBERT, which outperformed general multilingual BERT on code-mixed tasks; Indic models such as MuRIL [33] and IndicBERT [34] follow the same direction.

E. Sentiment Analysis of Code-Mixed Text

[35] Joshi et al. (2016) released a Hindi-English Facebook dataset and a sub-word LSTM that was 4–5% more accurate than traditional methods. [10] Patra et al. (2018) summarised the SAIL Code-Mixed shared task, which fixed the three-class formulation used here, and [36] Sarkar (2018) showed that multinomial Naive Bayes with n-grams placed third in it. [37] Patwa et al. (2020)

organised SemEval-2020 Task 9 (SentiMix), now the standard Hinglish benchmark, where most top systems fine-tuned multilingual transformers.

[38] Srinivasan (2020) fine-tuned multilingual BERT on Hinglish without special machinery, [39] Ma et al. (2020) combined XLM-R [40] with focal loss, and [41] Kumar et al. (2020) added self-attention to a CNN; these systems need GPUs and thousands of labelled tweets. [42] Singh and Lefever (2020) transferred English knowledge through cross-lingual embeddings, while [43] Baroi et al. (2020) reported an F-score of only 0.617 with a classical ensemble, confirming that Hinglish is hard for purely statistical models.

[44] Chakravarthi et al. (2020) created code-mixed Tamil-English and Malayalam-English [45] corpora, consolidated in DravidianCodeMix [46], and [47] Hande et al. (2020) a Kannada-English one; Hindi-English resources also exist for hate speech [48, 49]. [50] Mehmood et al. (2020) showed that unsupervised normalisation of Roman Hindi and Urdu spellings improves sentiment classification, and [51] Mabokela et al. (2023) concluded that lexicons and normalisation remain vital for under-resourced languages.

F. Emotion, Sarcasm, Explanation and Large Language Models

[52] Plutchik (1980) proposed eight basic emotions, the basis of the NRC Emotion Lexicon of [53] Mohammad and Turney (2013); [54] Demszky et al. (2020) released GoEmotions and [55] Acheampong et al. (2020) reviewed text emotion detection. [56] González-Ibáñez et al. (2011) showed that sarcasm is hard even for humans, and [57] Kumar et al. (2020) proposed an attention BiLSTM for it. [58] Ribeiro et al. (2016) introduced perturbation-based explanation, [59] Atanasova et al. (2020) evaluated explainability techniques, and [60] Ribeiro et al. (2020) used behavioural tests to expose negation failures in commercial sentiment models.

[61] Zhang et al. (2024) found that large language models handle simple polarity well but lag on fine-grained tasks such as aspect-based analysis [62], and [63] Zhang et al. (2023) showed that multilingual LLMs still underperform smaller fine-tuned models on code-switched data. LLMs also require network access, which conflicts with the privacy goal of this project.

Table 1. Literature review summary chart of the studies most relevant to this work (SN = serial number; the number in square brackets is the reference number).

SN	Name of Authors & Reference Number	Title of Paper	Objectives	Methodology	Outcome	Journal / Conference
1	Hutto & Gilbert (2014) [4]	VADER: A parsimonious rule-based model for sentiment analysis of social media text	Build a fast, general rule-based analyser for social-media text	Validated lexicon with five heuristics (punctuation, capitals, degree words, “but”, negation); compound score	Outperformed individual human raters on tweets; widely adopted baseline; English only	ICWSM (AAAI), 2014
2	Joshi et al. (2016) [35]	Towards sub-word level compositions for sentiment analysis of Hindi-English code mixed text	Classify sentiment of Hindi-English Facebook comments	New dataset; sub-word LSTM	4–5% above traditional methods and 18% above prior Hi-En system; needs labelled data	COLING, 2016

SN	Name of Authors & Reference Number	Title of Paper	Objectives	Methodology	Outcome	Journal / Conference
3	Patra et al. (2018) [10]	Sentiment analysis of code-mixed Indian languages: Overview of SAIL_Code-Mixed shared task	Benchmark three-class sentiment for Hi-En and Bn-En tweets	Shared task with released data and common evaluation	Nine teams participated; defines the three-class task used here	arXiv / ICON, 2018
4	Patwa et al. (2020) [37]	SemEval-2020 Task 9: Overview of sentiment analysis of code-mixed tweets	Create a benchmark for Hinglish and Spanglish sentiment	Large annotated tweet corpora; shared task	Top systems fine-tuned multilingual transformers; standard benchmark	SemEval (ICCL), 2020
5	Srinivasan (2020) [38]	MSR India at SemEval-2020 Task 9: Multilingual models can do code-mixing too	Test multilingual BERT on Hinglish sentiment	Fine-tuning mBERT without special code-mixing machinery	Competitive results; multilingual pre-training transfers to code-mixed text	SemEval, 2020
6	Baroi et al. (2020) [43]	NITS-Hinglish-SentiMix at SemEval-2020 Task 9	Classify Hinglish tweets with classical models	Ensemble of machine-learning classifiers	F-score 0.617; classical models struggle on Hinglish	SemEval, 2020
7	Mehmood et al. (2020) [50]	An unsupervised lexical normalization for Roman Hindi and Urdu sentiment analysis	Normalise spelling variants of Roman Hindi/Urdu	Unsupervised clustering of lexical variants	Normalisation improved sentiment classification	Information Processing & Management, 2020
8	Khanuja et al. (2020) [31]	GLUECoS: An evaluation benchmark for code-switched NLP	Unify evaluation of code-switched NLP tasks	Benchmark of tasks incl. Hi-En sentiment; mBERT baselines	Fine-tuning on synthetic code-switched data helps	ACL, 2020
9	Nayak & Joshi (2022) [32]	L3Cube-HingCorpus and HingBERT	Provide Hinglish pre-training data and models	Large unlabelled corpus; BERT pre-training	HingBERT beat general multilingual BERT on code-mixed tasks	WILDRE-6 @ LREC, 2022
10	Chakravarthi et al. (2022) [46]	DravidianCodeMix: Sentiment analysis and offensive language identification dataset	Release code-mixed Dravidian sentiment data	Annotated YouTube comments; ML and DL baselines	Class imbalance and annotation effort are key challenges	Language Resources and Evaluation, 2022
11	Catelli et al. (2022) [20]	Lexicon-based vs. BERT-based sentiment analysis: A comparative study in Italian	Compare lexicon and transformer methods	Lexicon analyser vs fine-tuned BERT	BERT more accurate; lexicon faster, data-free and interpretable	Electronics (MDPI), 2022
12	Ribeiro et al. (2020) [60]	Beyond accuracy: Behavioral testing of NLP models with CheckList	Test model behaviour beyond accuracy	Capability-based test suites (e.g., negation)	Found critical failures in commercial sentiment models	ACL, 2020
13	Mabokela et al. (2023) [51]	Multilingual sentiment analysis for under-resourced languages: A systematic review	Map research on low-resource sentiment analysis	Systematic literature review	Lexicons and normalisation remain vital where data are scarce	IEEE Access, 2023

SN	Name of Authors & Reference Number	Title of Paper	Objectives	Methodology	Outcome	Journal / Conference
14	Zhang et al. (2023) [63]	Multilingual large language models are not (yet) code-switchers	Evaluate LLMs on code-switching tasks	Benchmarking multilingual LLMs vs fine-tuned models	LLMs underperform smaller fine-tuned models on code-switched data	EMNLP, 2023
15	Zhang et al. (2024) [61]	Sentiment analysis in the era of large language models: A reality check	Assess LLMs on sentiment tasks	Evaluation on 13 tasks with zero/few-shot prompting	Good on simple polarity, weaker on fine-grained tasks	Findings of NAACL, 2024

G. Research Gap

Widely used rule-based tools are English-only; the strongest code-mixed systems need large labelled corpora, GPUs and often cloud access; few systems explain verdicts at the word level; and small studies rarely report confidence intervals, significance tests or ablations. Text Sentiment Analyser addresses these gaps with a lightweight, explainable, privacy-preserving hybrid and a statistically careful evaluation.

Methods and Materials:

A. System Overview and Architecture

The overall architecture appears in Fig. 1. Text can be entered from the keyboard, dictated through a voice-input widget or uploaded in bulk as a CSV file. A language identifier first tags the input as English, Roman Hindi, Hinglish or Devanagari Hindi, the last of which is flagged as unsupported. A normaliser then shortens runs of repeated letters and maps spelling variants to canonical forms. The normalised token stream is passed in parallel to the rule engine and to the machine-learning engines, and a fusion stage combines their probabilities into a final verdict with a confidence value. Separate modules reuse the same tokeniser to compute scores for eight emotions, aspect-level sentiment, a sarcasm and mixed-signal indicator, a leave-one-out explanation and descriptive text statistics. The pipeline consists of roughly 4,300 lines of JavaScript spread over 22 files; it runs without modification in a browser or under Node.js, and a 158-line Python script reproduces the machine-learning experiments with scikit-learn [64].

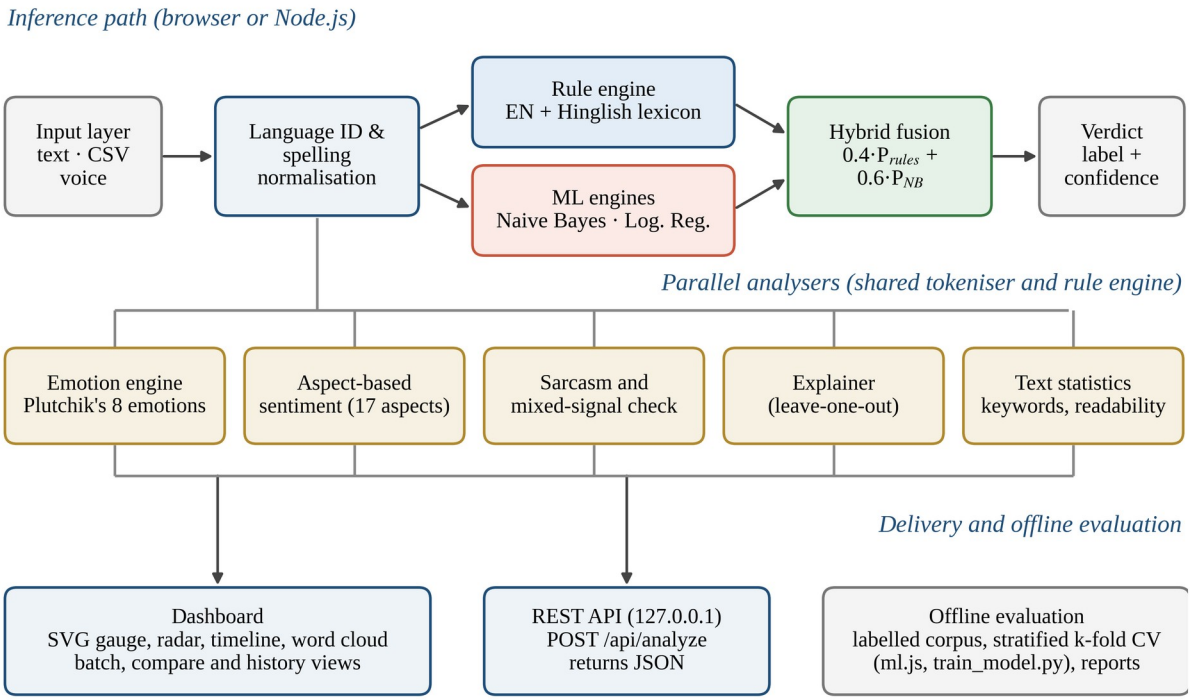


Fig. 1. Architecture of Text Sentiment Analyser. The inference path (top) feeds a set of parallel analysers (middle) whose outputs reach the user through the dashboard and a local REST API; the offline evaluation component (bottom right) reuses the same engines.

B. Pilot Corpus

Evaluation uses a pilot corpus of 210 short English and Hinglish texts that the project team wrote in the style of public comments about films, phones, cricket, food, travel, college life and customer service (Table 2). Every text received one of three labels, Positive, Neutral or Negative, according to the project's labelling guide. Under that guide, the dominant or concluding opinion decides a mixed comment, the intended rather than the literal meaning decides a sarcastic one, and factual statements and questions are Neutral. The three classes are balanced, the average text is 7.1 words long, and the language identifier assigns 133 texts to Hinglish, 63 to Roman Hindi and 14 to English. No user names or other personal data were collected.

Two properties of the corpus restrict the conclusions that can be drawn, and they are stated here at the start. First, the corpus was created to exercise the pipeline, and the Hinglish word list was compiled at the same time. The rule-based engine is therefore tested on material it was developed with, so its accuracy is an optimistic upper bound and not an estimate of performance on unseen social-media text. Second, agreement between independent annotators was not measured. The learned models, by contrast, are always tested on held-out folds.

Table 2. Statistics of the pilot corpus.

Property	**Value**
Texts	210 (English and Hinglish, Roman script)
Class distribution	Positive 71, Neutral 69, Negative 70
Domains (texts)	movies 42, phones 32, cricket 29, food 29, general 23, college 22, travel 21, service 12

Property	Value
Length	mean 7.1 words (range 3–13); 12 texts contain emoji
Detected language	Hinglish 133, Roman Hindi 63, English 14
Provenance	Written by the project team; no scraped user data

C. Language Identification and Normalisation

Language identification. The detector counts how many tokens belong to a list of 211 frequent Romanised-Hindi words, covering function words and common opinion words, and how many belong to a short list of English function words. A text is tagged Roman Hindi when at least 55% of its words are Hindi and no more than 15% are English, and Hinglish when the Hindi share reaches 15% (or when it contains at least two Hindi words and a Hindi share of 10%). Devanagari input is recognised from its Unicode range. Words that are ambiguous between the two languages, such as *the*, *me*, *to*, *so* and *par*, are excluded from the Hindi list.

Spelling normalisation. Every alphabetic token is lower-cased, any run of three or more identical letters is reduced to two (*bahuuut* → *bahuut*), and a dictionary of 94 entries maps common variants to a canonical spelling (for example *bhot*, *bahot*, *bohot* → *bahut*; *nhi*, *nahin* → *nahi*; *badiya*, *bdiya* → *badhiya*). The normalised tokens are used to build the machine-learning features. The rule engine instead lists frequent variants explicitly in its lexicon, so that every word it matches can be displayed to the user unchanged. The sizes of all linguistic resources are given in Table 3.

Table 3. Linguistic resources used by the system.

Resource	Size	Notes
English sentiment lexicon	292 entries	Weights 1–4; positive and negative
Hinglish sentiment lexicon	184 entries	103 positive, 81 negative; spelling variants listed
Hindi intensifiers and dampeners	28 entries	For example bahut ×1.6, ekdum ×1.7, thoda ×0.6
English intensifiers and dampeners	27 entries	For example very, extremely, slightly
Emoji and emoticon scores	Signed, −4 to +4	Hand-assigned; cf. emoji sentiment ranking [65]
Spelling-variant dictionary	94 mappings	ML features and language identification
Romanised-Hindi vocabulary	211 words	Language identification
Emotion lexicon	376 word–emotion entries	Eight Plutchik emotions plus an emoji map
Aspect categories	17	Keyword lists, for example camera, battery, price, acting

D. Rule-Based Hinglish Engine

The rule engine, summarised in Algorithm 1 and Table 4, scans the tokens once from left to right. A tokeniser first separates words, emoji, emoticons and punctuation. During the scan the engine maintains a small amount of state, namely a pending negation counter, a pending booster factor, a contrast flag and a record of the most recent sentiment-bearing word, and it accumulates positive evidence in P and negative evidence in N.

Table 4. Parameters of the rule engine.

Parameter	Value	Role
Valence range	1–4 (emoji −4 to +4)	Lexicon entries

Parameter	Value	Role
Negation factor	−0.75	English and Hindi negation
Negation window	3 tokens	English forward window; Hindi look-back and fallback
Intensifier factors	0.5–1.8 (EN), 0.6–1.7 (HI)	Applied to the next sentiment word
Contrast words	×0.5 before, ×1.5 after	but, however, lekin, magar
Capitalised words	×1.3	Words longer than two letters
Exclamation marks	+8% each, at most four	Amplify the dominant polarity
Normalisation constant	15	Eq. (2)
Decision threshold	|c| ≥ 0.05	Together with P > N or N > P

Algorithm 1. Hinglish rule-based sentiment scoring (simplified from the implementation).

Input: text T; flag H (Hinglish layer enabled). **Output:** label and compound score c.

tokens ← tokenise(T); P ← 0; N ← 0; boost ← 1; neg ← 0; contrast ← false; last ← none
for each token t in tokens **do**
  if t is clause punctuation **then** neg ← 0; boost ← 1; last ← none; **continue**
  w ← lower-case(t) with repeated letters collapsed
  if w is a contrast word **then** P ← 0.5·P; N ← 0.5·N; contrast ← true; reset neg, boost, last; **continue**
  if H **and** w is a Hindi post-negator (nahi, nhi, nahin, nai) **then**
    if an un-negated hit last lies within 3 words **then** replace its score s by −0.75·s (move evidence between P and N)
    else neg ← 3; **continue**
  if w is an English negator, an n't contraction or a Hindi pre-negator **then** neg ← 3; **continue**
  if w is a booster that is not itself a sentiment word **then** boost ← boost × factor(w); **continue**
  v ← signed valence of t (English lexicon; Hinglish lexicon if H; emoji; emoticon)
  if v = 0 **then** (**if** neg > 0 **then** neg ← neg − 1); **continue**
  s ← v · boost; **if** t is ALL-CAPS **then** s ← 1.3·s
  if neg > 0 **and** t is a word (emoji are never negated) **then** s ← −0.75·s
  if contrast **then** s ← 1.5·s
  add |s| to P (s > 0) or to N (s < 0); last ← (t, s); boost ← 1; neg ← 0
amplify the larger of P and N by 1 + 0.08·min(number of "!", 4)
c ← (P − N) / √((P − N)² + 15)
return Positive if c ≥ 0.05 and P > N; Negative if c ≤ −0.05 and N > P; otherwise Neutral

Lexicon and emoji. Every lexicon word carries an integer valence between 1 and 4, and emoji and emoticons carry signed scores between −4 and +4. The English and Hinglish entries are consulted together; switching off the Hinglish layer turns the same engine into the English-only baseline.

Negation. An English negator opens a window of three tokens within which sentiment words are reversed and attenuated by the factor −0.75, following the idea of shift negation [5]. Because a Hindi negator follows the word it negates, the engine, on reaching one, searches back up to three words for the most recent sentiment word that has not already been negated and multiplies its score by −0.75, so that *acchi nahi thi* becomes negative. Only when no such word exists does it open a forward window, as in *nahi pasand* (not liked). Emoji are never reversed, so *help nahi ki* 😡 (did not help) stays negative.

Intensifiers and contrast. A booster scales the next sentiment word (*bahut* ×1.6, *ekdum* ×1.7, *thoda* ×0.6). When a contrast word (*but*, *however*, *lekin*, *magar*) appears, the evidence gathered so

far is halved and all later evidence is multiplied by 1.5. This reflects the observation that the clause following a contrast usually states the writer's final opinion [4].

Emphasis and decision. Words written fully in capitals are multiplied by 1.3, and each exclamation mark strengthens the dominant polarity by 8%, up to four marks. The score of a single token is given by Eq. (1), and Eq. (2) maps the net evidence to a compound score in the interval (−1, 1), following the normalisation used by VADER [4].

$$s_t = v_t \cdot b_t \cdot \eta_t \cdot \kappa_t \quad (1)$$

$$c = x / \sqrt{(x^2 + 15)}, \quad x = P - N \quad (2)$$

In these equations v_t is the lexicon valence of token t, b_t the booster factor, η_t equals −0.75 for a negated word and 1 otherwise, and κ_t equals 1.5 after a contrast word and 1 otherwise. All constants are listed in Table 4.

E. Machine-Learning Classifiers

The two classifiers share one feature set: normalised unigrams and bigrams of adjacent tokens. Emoji and the marks "!" and "?" are kept as tokens, because punctuation carries useful information in short comments.

Naive Bayes. A multinomial Naive Bayes model [66] computes $P(y \mid d) \propto P(y) \prod P(f \mid y)^{n(f,d)}$, where the feature likelihoods are estimated with Laplace smoothing as in Eq. (3). The implementation uses $\alpha = 1$ and also applies add-one smoothing to the class priors.

$$P(f \mid y) = (n_{f,y} + \alpha) / (N_y + \alpha|V|) \quad (3)$$

Here $n_{f,y}$ is the number of occurrences of feature f in class y, N_y is the total feature count of class y, and $|V|$ is the size of the vocabulary.

Logistic regression. A multinomial logistic regression model is trained on TF-IDF vectors [67, 68] that use sub-linear term frequency and L2 normalisation (Eq. 4). The model has a softmax output (Eq. 5) and is optimised by per-example stochastic gradient descent [69] with L2 regularisation (Eq. 6), using 40 epochs, an initial learning rate $\eta_0 = 0.4$ that decays as $\eta_e = \eta_0/(1 + 0.05e)$, $\lambda = 10^{-4}$ and a fixed random seed.

$$w_{f,d} = (1 + \ln tf_{f,d}) \cdot idf_f, \quad idf_f = \ln((1 + N)/(1 + df_f)) + 1 \quad (4)$$

$$p_k = \exp(z_k) / \Sigma_j \exp(z_j), \quad z_k = W_k \cdot x + b_k \quad (5)$$

$$W_k \leftarrow W_k - \eta_e [(p_k - y_k) x + \lambda W_k] \quad (6)$$

Reference implementations. To check the hand-written models against a standard library, the script train_model.py fits scikit-learn pipelines on the same normalised text. A TF-IDF vectoriser with unigrams, bigrams and sub-linear term frequency feeds three models: multinomial Naive Bayes ($\alpha = 0.5$), logistic regression ($C = 5$) and a linear support vector machine ($C = 1$) [70].

F. Hybrid Fusion

The hybrid model turns the compound score c of the rule engine into a pseudo-probability vector (Eq. 7) and mixes it with the posterior of the Naive Bayes model (Eq. 8); the predicted label is the class with the highest P_{hybrid}. The mixing weight $w = 0.4$ is a fixed design constant of the project, and its sensitivity is examined in the section "Results". By construction, $P_{rules}(Pos)$ exceeds $P_{rules}(Neu)$ only when $c > 0.5$, a property that becomes important in the discussion.

$$P_{rules}(Pos) = \max(c, 0), \quad P_{rules}(Neg) = \max(-c, 0), \quad (7)$$

$$P_{rules}(Neu) = 0.9 \cdot \max(0, 1 - P_{rules}(Pos) - P_{rules}(Neg)) + 0.05$$

$$P_{hybrid}(y) = w \cdot P_{rules}(y) + (1 - w) \cdot P_{NB}(y), \quad w = 0.4 \tag{8}$$

G. Emotion, Aspect, Sarcasm and Explanation Modules

The emotion module is lexicon-based. A total of 376 English and Hinglish word–emotion entries (joy 64, trust 47, fear 45, surprise 34, sadness 53, disgust 42, anger 55, anticipation 36), together with an emoji map, assign evidence to Plutchik's eight basic emotions [52]. Negated words are ignored, and the emotion with the most evidence is reported as dominant. The aspect module divides a text into clauses at punctuation and contrast words, searches each clause for keywords belonging to 17 aspects (such as camera, battery, price, delivery, acting and songs) and scores each aspect with the rule engine, producing verdicts such as "camera: positive, battery: negative" [62].

The sarcasm module is a rule-based indicator rather than a trained classifier. It accumulates evidence from eye-roll and clap emoji, ironic phrase patterns (*oh great*, *wah kya baat hai*), praise words placed next to words describing mishaps (*late*, *crash*) and emoji that contradict the words, and it reports Unlikely, Possible or Likely together with its reasons. Finally, the explainer attributes the verdict to individual words by leave-one-out perturbation: deleting token t_i and rescoring the text yields its impact Δ_i (Eq. 9). This also reveals the effect of function words such as *nahi*, *bahut* and *but*, in the spirit of perturbation-based explanation [58, 59].

$$\Delta_i = c(T) - c(T \setminus t_i) \tag{9}$$

H. Software Implementation and Privacy

The user interface provides a live analyser (with a gauge, an emotion radar, a sentence timeline and a word cloud), batch CSV processing with export, a side-by-side comparison view, a history dashboard and an evaluation page that runs the five-engine cross-validation on either the bundled corpus or an uploaded one. All charts are drawn with hand-written SVG, so the application depends on no third-party library. A dependency-free Node.js server delivers the front end and exposes POST /api/analyze, which returns the label, compound score, confidence, emotions, sentences, keywords and statistics as JSON. The server listens on 127.0.0.1 only, rejects request bodies larger than 20 KB and texts longer than 5,000 characters, validates JSON input and blocks path traversal. A content-security policy prevents the page from sending requests to outside servers, and analysis history is kept in the browser. An automated test script checks the engines, the machine-learning module and the HTTP interface.

I. Algorithmic Complexity

Let n be the number of tokens in a text, $L \approx 2n - 1$ the number of unigram and bigram features, N the number of training texts, $K = 3$ the number of classes and E the number of epochs. Lexicon, booster and negator lookups are constant-time hash-map operations, so tokenisation, language identification and rule scoring are linear, $O(n)$. Naive Bayes training costs $O(N \cdot L)$ and prediction $O(K \cdot L)$; logistic regression training costs $O(E \cdot N \cdot L \cdot K)$. The leave-one-out explainer rescores the text once per token and is therefore $O(n^2)$, so it is capped at 400 tokens.

J. Experimental Protocol and Metrics

All JavaScript engines are evaluated by stratified k-fold cross-validation [71] with $k = 5$. The texts of each class are shuffled with a seeded generator and dealt round-robin into folds, so every fold preserves the class balance; models are trained on four folds and tested on the fifth. The rule-based

engines have no trainable parameters, so for them the folds merely partition the evaluation. The primary run uses seed 7, and the whole procedure is repeated with seeds 7, 11, 21, 42 and 99 to measure sensitivity to the split. The Python experiments use five folds with random state 42; their splits therefore differ from those of the JavaScript runs, and the two sets of results are not paired.

The metrics are accuracy; macro-averaged precision, recall and F1 (Eq. 10); Cohen's kappa between predictions and gold labels [72] (Eq. 11); and 95% confidence intervals obtained from 2,000 bootstrap resamples of the 210 pooled out-of-fold predictions [73]. Paired differences between engines are assessed with McNemar's test with continuity correction [74, 75] (Eq. 12), where b and c count the texts that only one of the two engines classifies correctly. Kappa values are interpreted with the bands of Landis and Koch [76]. Additional experiments remove one component of the rule engine at a time, sweep the hybrid weight w and the smoothing constant α, and train on growing fractions of the training folds. No setting was tuned to improve the headline results; the sweeps are reported purely as sensitivity analyses. The software environment is listed in Table 5.

$$F1_{macro} = (1/3)\,\Sigma_c\, 2 \cdot P_c \cdot R_c / (P_c + R_c) \quad (10)$$

$$\kappa = (p_o - p_e) / (1 - p_e) \quad (11)$$

$$\chi^2 = (|b - c| - 1)^2 / (b + c), \quad df = 1 \quad (12)$$

Table 5. Experimental setup.

Item	Setting
Runtime	Node.js v22.22.2 (JavaScript engines, benchmarks); Python 3.12.3 with scikit-learn 1.8.0, NumPy and Matplotlib [77, 78]
Cross-validation	Stratified, k = 5; seeds 7, 11, 21, 42, 99 (JavaScript); random state 42 (Python)
Bootstrap	2,000 resamples of the pooled predictions, seed 123
Naive Bayes	α = 1 (JavaScript); α = 0.5 (scikit-learn)
Logistic regression	40 epochs, $\eta_0 = 0.4$, $\lambda = 10^{-4}$ (JavaScript); C = 5 (scikit-learn)
Linear SVM	C = 1 (scikit-learn)
Hybrid	Rule weight w = 0.4; Naive Bayes α = 1
Hardware	Shared cloud Linux container; timings are indicative only

The next section presents the measurements obtained with this protocol.

Results:

A. Overall Performance

The main comparison is reported in Table 6 and Fig. 2. The English-only lexicon reaches 55.2% accuracy and 52.8% macro-F1. Because it does not recognise most Hindi opinion words, it labels 160 of the 210 texts (76%) as Neutral, and its recall is only 35.2% for the Positive class and 32.9% for the Negative class. Enabling the Hinglish layer lifts accuracy to 92.4% (95% CI 88.6–95.7%) and kappa to 0.89, an improvement of 37.2 points that no other component comes close to. The trained classifiers lie between these two extremes. On the primary split Naive Bayes reaches 70.0% and logistic regression 67.6%, but both average 65.9% across the five splits, and the gap between

them is smaller than the variation from one split to the next. The hybrid reaches 83.8% on the primary split and 82.1% on average, which is 16.2 points above Naive Bayes on its own.

Table 6. Performance of the five engines under stratified five-fold cross-validation (n = 210). Columns 2–5 refer to the primary split (seed 7); the last column averages five splits.

Engine	Accuracy, fold mean ± SD (%)	Macro-F1, fold mean ± SD (%)	95% CI of pooled accuracy (%)	Cohen's κ	Accuracy, five-split mean ± SD (%)
English lexicon (baseline)	55.3 ± 6.4	52.8 ± 8.0	48.6–61.9	0.33	55.2 ± 0.0
Hinglish-aware rules (ours)	92.4 ± 2.4	92.4 ± 2.2	88.6–95.7	0.89	92.4 ± 0.0
Naive Bayes	70.0 ± 3.1	69.9 ± 3.6	63.8–75.7	0.55	65.9 ± 2.8
Logistic regression	67.6 ± 2.8	67.3 ± 2.9	61.4–73.3	0.51	65.9 ± 1.8
Hybrid: rules + NB (ours)	83.8 ± 4.7	83.7 ± 4.6	78.6–88.6	0.76	82.1 ± 1.6

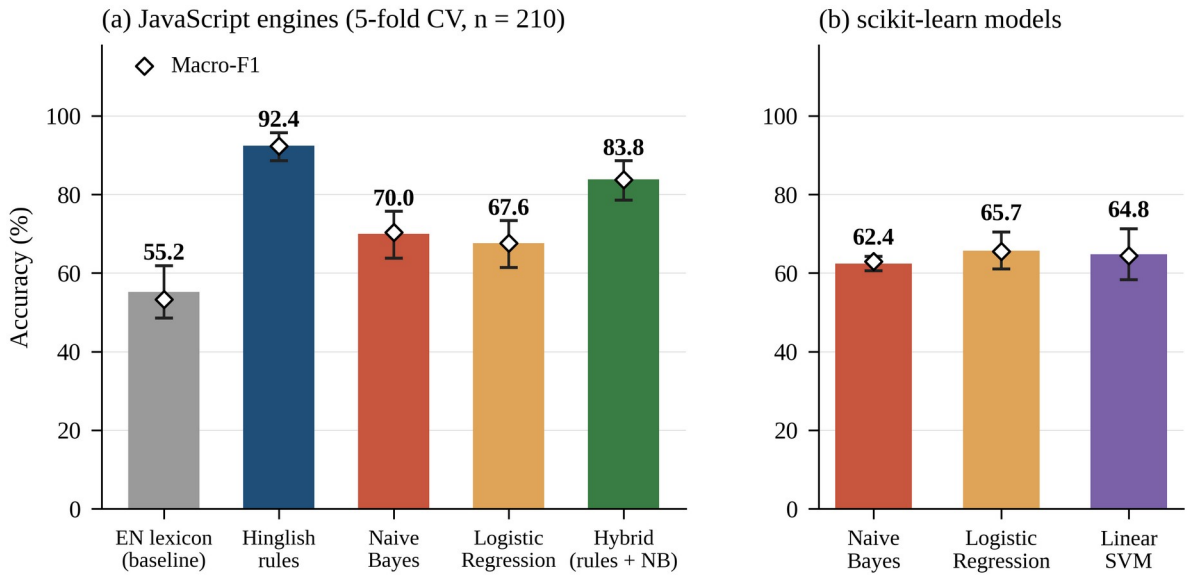


Fig. 2. Accuracy of the compared engines. (a) JavaScript engines on the primary split (bars: pooled accuracy with 95% bootstrap interval; diamonds: macro-F1). (b) scikit-learn models (mean ± SD over five folds; diamonds: macro-F1).

The scikit-learn reference models in Table 7 show the same picture. Naive Bayes, logistic regression and the linear SVM reach 62.4%, 65.7% and 64.8% accuracy respectively, with fold standard deviations between 1.8 and 6.5 points. Two independent implementations of the same model families thus agree to within about four points (comparing the five-split JavaScript means with the Python run), and both stay far below the rule engine. This suggests that, with only about 170 training texts, the learned models are limited by the amount of data rather than by the choice of algorithm.

Table 7. Results of the scikit-learn models (five-fold cross-validation, macro-averaged).

Model	Accuracy (%)	Precision (%)	Recall (%)	Macro-F1 (%)
Multinomial Naive Bayes	62.4 ± 1.8	64.4	62.5	63.0 ± 1.3
Logistic regression	65.7 ± 4.7	65.9	65.9	65.5 ± 4.7
Linear SVM	64.8 ± 6.5	64.5	65.0	64.4 ± 6.6

B. Statistical Reliability

With 210 texts the bootstrap intervals are wide, with half-widths between 3.6 and 6.7 points, so only large differences can be trusted. On the Landis–Koch scale [76], agreement with the gold labels is fair for the baseline ($\kappa = 0.33$), moderate for Naive Bayes and logistic regression (0.55 and 0.51), substantial for the hybrid (0.76) and almost perfect for the Hinglish engine (0.89). McNemar tests on the pooled predictions (Table 9) confirm that the Hinglish engine outperforms the English baseline (81 texts are correct only for the former and 3 only for the latter, $p < 0.0001$), and that the hybrid outperforms both Naive Bayes (30 against 1) and logistic regression (35 against 1), each with $p < 0.0001$. The rule engine is also better than the hybrid (24 against 6, $p = 0.002$). Naive Bayes and logistic regression cannot be distinguished (9 against 4, $p = 0.27$). Across the five splits the accuracy of Naive Bayes ranges from 61.5% to 70.0%, a spread larger than its difference from logistic regression, so the ranking of the learned models on a 210-text corpus should not be over-interpreted.

C. Class and Domain Behaviour

The per-class scores in Table 8 follow a consistent pattern. Apart from the English baseline, every engine recognises Neutral texts well (F1 0.87–0.92), because the neutral comments in the corpus carry clear cues such as question marks and time expressions (*baje*, *kal*, *abhi*), which Naive Bayes also ranks as its most indicative Neutral features. The learned models, however, confuse the two polar classes. Naive Bayes labels 23 positive texts as negative and 23 negative texts as positive, and logistic regression makes these errors 24 and 26 times respectively (Fig. 3). The reason is that many opinion words occur only once or twice in the training folds, so an unseen word of praise or complaint contributes nothing at test time. The Hinglish engine has the opposite weakness: 8 of its 10 errors on Negative texts are predictions of Neutral, mostly for complaints that contain no lexicon word. Across domains the Hinglish engine is the best or joint-best engine everywhere. Its lowest score is in the phones domain (84%), where complaints are often implicit, and in the 12-text service domain all engines except Naive Bayes tie at 67%.

Table 8. Per-class precision (P), recall (R) and F1 of the five engines (pooled out-of-fold predictions, primary split).

Engine	Pos P	Pos R	Pos F1	Neu P	Neu R	Neu F1	Neg P	Neg R	Neg F1
English lexicon (baseline)	1.00	0.35	0.52	0.42	0.99	0.59	0.92	0.33	0.48
Hinglish-aware rules	0.96	0.94	0.95	0.87	0.97	0.92	0.95	0.86	0.90
Naive Bayes	0.62	0.63	0.63	0.89	0.86	0.87	0.61	0.61	0.61
Logistic regression	0.58	0.59	0.58	0.86	0.88	0.87	0.59	0.56	0.57
Hybrid: rules + NB	0.78	0.82	0.80	0.89	0.94	0.92	0.84	0.76	0.80

D. Ablation and Sensitivity

Removing the components of the rule engine one at a time (Table 10) shows that the Hindi lexicon is by far the most important element: without it accuracy drops from 92.4% to 55.2%, and only 5 of the 12 targeted test sentences are classified correctly. The remaining components change corpus accuracy by one point at most (emoji and emoticon scores by 1.0 point, intensifiers by 0.5 points, and Hindi post-negation, the contrast rule and English pre-negation not at all). Even so, removing

them breaks 3, 2 and 2 of the 12 targeted sentences respectively, including *movie acchi nahi thi* and *camera mast hai but battery bekaar*. The corpus is therefore a weak test of these mechanisms, since only 20 texts contain a Hindi negator, 2 a contrast word and 2 an English negator, an observation in line with the case for targeted behavioural testing [60].

The unchanged accuracy also hides the fact that post-negation does alter predictions: it changes eight of them, correcting four (for example *Bilkul pasand nahi aayi movie*) and breaking four. Two of the broken cases are "never … such" idioms built on *kabhi nahi*, in which a negator together with a superlative expresses praise or censure rather than denial; one is a comparative (*isse best phone nahi milega*); and one negates the neutral noun *problem*. For the learned model, spelling normalisation adds 1.1 points to Naive Bayes (from 64.8% to 65.9%), removing bigrams changes accuracy by +0.4 points (66.3% with unigrams only), and the smoothing constant has little effect (from 64.1% at $\alpha = 0.1$ to 67.2% at $\alpha = 2$). Each of these differences is smaller than the standard deviation of 1.9–3.0 points across splits, so none of them is established.

Fig. 4a varies the rule weight w of Eq. (8). Accuracy rises almost steadily from 65.9% (Naive Bayes only) to a peak of 89.4% at $w = 0.7$ and then falls to 87.1% at $w = 1$, so the project's default of 0.4 (82.1%) is not the best value on this corpus. Selecting w on the same folds used for testing is optimistic, so 89.4% is an upper bound unless w is chosen by nested validation. The value at $w = 1$ is lower than the 92.4% of the direct rule label because of Eq. (7): the rule probability of Positive or Negative exceeds that of Neutral only when $|c| > 0.5$, while the direct rule uses a threshold of 0.05, so the probabilistic form pushes weakly polarised texts towards Neutral.

E. Data Efficiency

The learning curves in Fig. 4b show where fusion helps most. With only 17 training texts, Naive Bayes and logistic regression score 49.9% and 51.3%, about 17 points above the chance level for three balanced classes, whereas the hybrid already reaches 75.5% because its rule component brings prior knowledge. With the full 168 training texts per split, the learned models reach 67.0% and the hybrid 80.4%. The learned models are still gaining about seven points for every doubling of the data (60.2% at 84 texts and 67.0% at 168), while the hybrid gains only 1.6 points over the same step and stays 11 points below the rule engine (91.5% on the same test folds).

F. Error Analysis

The 16 errors of the Hinglish engine (Table 11) fall into three groups. Nine involve implicit or unlexicalised sentiment: complaints such as *Battery bahut jaldi khatam ho jaati hai* (the battery drains very fast) or *Wifi kabhi kaam nahi karta campus mein* contain no word with a stored polarity, so the engine answers Neutral; in one of these cases the cause is the spelling variant *dheemi*, which is missing from the lexicon. Four are errors of negation scope, three of which involve the *kabhi nahi* construction or a comparative negation. Three arise because the polarity of a word depends on its domain: *problem* in *problem solve kar di*, *jeet* (win) in a neutral report of a cricket toss, and *refund* in a neutral statement of policy. Because the engine has no trainable parameters, exactly the same 16 errors occur on the full corpus and in the pooled out-of-fold predictions.

The errors of the learned models are dominated by unseen vocabulary. On the primary split, Naive Bayes labels *Heroine ki acting lajawab thi* (the heroine's acting was superb) and *Flight staff bahut helpful the* as Negative, because *lajawab* never occurs in the training folds and the remaining tokens carry little information. The most indicative features of the model include genuine cues

(*mast*, *khush*, *bekaar*, *ghatiya*) alongside artefacts of the corpus (*bilkul*, *sab*, *baar*, *itna*, *main*) that would not generalise. The hybrid inherits the errors of the rules and adds some learned ones; for example, it labels *Goa trip bahut mazedaar raha* Negative although the rule engine is correct. Naive Bayes posteriors tend to be extreme, which probably allows a confident but wrong vote to outweigh a correct rule score at w = 0.4.

G. Emotion, Aspect and Explanation Outputs

The corpus has no gold emotion labels, so only the distribution of the dominant emotion is reported and no accuracy is claimed. Joy dominates 39% of the positive texts, while negative texts show anger (21%), fear (14%), disgust (11%) and sadness (6%), proportions that are plausible for these classes. Neutral texts, however, show surprise in 28% of cases, and inspection uncovered a defect: 14 of the 19 neutral texts labelled surprise are triggered by the postposition *mein* (“in”), which entered the emotion lexicon when the phrase *sach mein* (really) was split into separate words, and five by the interrogative *kya*. The aspect module returns “camera: positive” and “battery: negative” for *camera mast hai but battery bekaar*. For *movie acchi nahi thi but songs mast the* the engine returns a compound score of +0.70 (Positive), and the explainer attributes the largest impact to *mast* (+0.89), followed by the contrast word *but* (+0.34), while *nahi* and *acchi* contribute small negative amounts.

H. Runtime and Functional Testing

The automated test script passes completely. It covers the English engine cases, the Hinglish checks, the machine-learning module including a cross-validation sanity check, and the HTTP interface, including malformed JSON (HTTP 400) and rejection of path traversal. Table 12 lists the twelve targeted sentences used in the ablation. Analysing one text with the full pipeline takes 0.12 ms on average (0.013 ms for the rule engine alone, and 0.029 ms and 0.015 ms per prediction for Naive Bayes and logistic regression), and training on all 210 texts takes 2.4 ms for Naive Bayes and 53 ms for logistic regression. Scaling experiments agree with the complexity analysis: an eight-fold increase in input length multiplies the time of the rule engine by 9.3, the same step multiplies Naive Bayes training time by 7.7, and a 3.9-fold increase in input length multiplies the explainer’s time by 13.2, close to the quadratic expectation of 15. These timings come from one Node.js run in a cloud container and include start-up noise at small sizes, so growth ratios are more informative than absolute times.

Figures and Tables:

The exhibits below extend the results reported above; each one is referred to in the text.

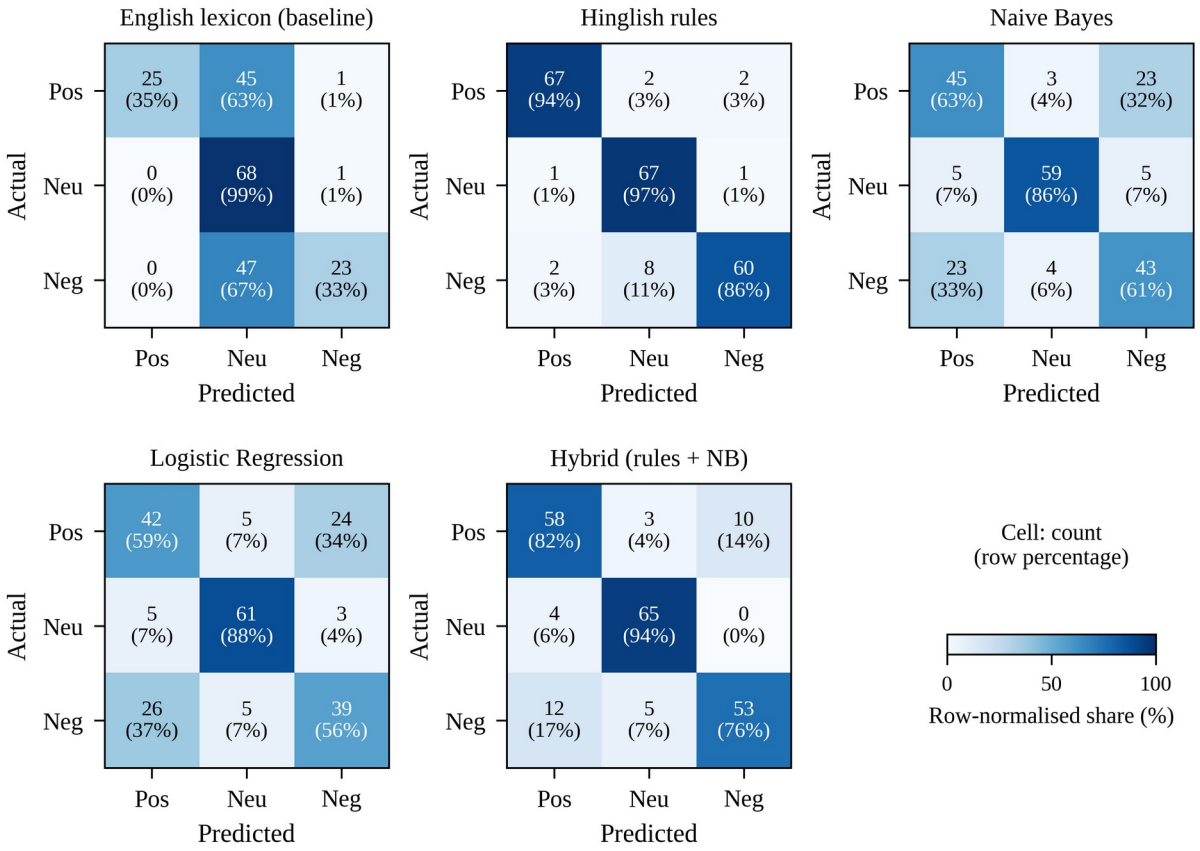


Fig. 3. Confusion matrices of the five engines (pooled out-of-fold predictions, primary split; rows are actual classes).

Table 9. McNemar tests on pooled out-of-fold predictions. The two count columns give the number of texts classified correctly only by the first and only by the second engine.

Comparison	First only	Second only	χ^2	p
Hinglish rules vs English baseline	81	3	70.58	< 0.0001
Hinglish rules vs Hybrid	24	6	9.63	0.0019
Hinglish rules vs Naive Bayes	53	6	35.86	< 0.0001
Hybrid vs Naive Bayes	30	1	25.29	< 0.0001
Hybrid vs Logistic regression	35	1	30.25	< 0.0001
Naive Bayes vs Logistic regression	9	4	1.23	0.2673

Table 10. Ablation of the Hinglish rule engine. Δ is the change in accuracy relative to the full engine, in percentage points; the last column counts how many of the 12 targeted sentences of Table 12 are classified correctly.

Variant	Accuracy (%)	Macro-F1 (%)	Δ	Texts changed	Unit tests passed
Full Hinglish engine	92.4	92.3	—	0	12/12
− Hindi post-negation	92.4	92.3	0.0	8	9/12
− Contrast rule (but, lekin)	92.4	92.3	0.0	0	10/12
− Intensifiers (bahut, ekdum)	91.9	91.9	−0.5	1	12/12
− English pre-negation	92.4	92.3	0.0	0	10/12

Variant	Accuracy (%)	Macro-F1 (%)	Δ	Texts changed	Unit tests passed
− Emoji and emoticon scores	91.4	91.4	−1.0	2	11/12
− Hindi lexicon (English only)	55.2	53.3	−37.1	86	5/12

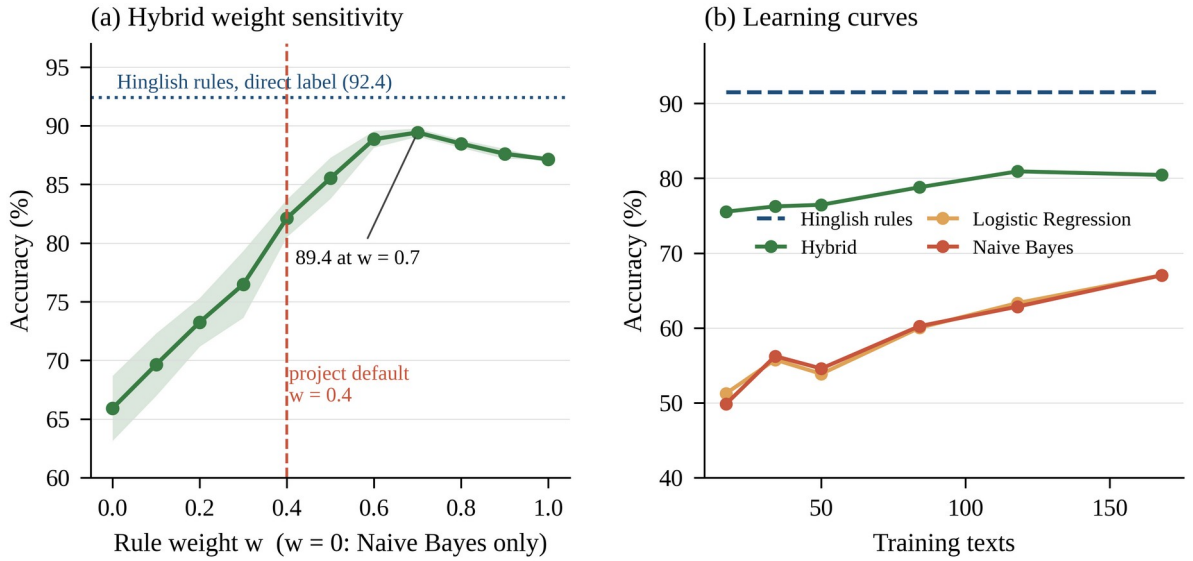


Fig. 4. Sensitivity analyses: (a) accuracy of the hybrid as the rule weight w varies (mean ± SD over five splits); (b) learning curves of the learned models, the hybrid and the rule engine (mean over ten train–test splits).

Table 11. The 16 misclassified texts of the Hinglish engine with gold label, prediction and error category.

#	Text	Gold	Pred.	Category
1	Bhai yeh web series toh next level hai	Pos	Neu	Implicit / unlexicalised
2	Is price mein isse best phone nahi milega	Pos	Neg	Negation scope
3	Itna swadisht paneer kabhi nahi khaya	Pos	Neg	Negation scope (kabhi nahi)
4	Customer care ne turant problem solve kar di, thanks	Pos	Neu	Domain-dependent polarity
5	Itna wahiyat climax kabhi nahi dekha	Neg	Pos	Negation scope (kabhi nahi)
6	Battery bahut jaldi khatam ho jaati hai	Neg	Neu	Implicit / unlexicalised
7	Charger kaam nahi kar raha, paise doob gaye	Neg	Neu	Implicit / unlexicalised
8	Speaker ki awaaz bahut dheemi hai	Neg	Neu	Implicit / unlexicalised (variant)
9	Customer care se baat karo toh koi jawab nahi deta	Neg	Neu	Implicit / unlexicalised
10	Pizza ka base jala hua tha	Neg	Neu	Implicit / unlexicalised
11	Bus mein itni bheed thi ki saans lena mushkil tha	Neg	Neu	Implicit / unlexicalised
12	Wifi kabhi kaam nahi karta campus mein	Neg	Neu	Implicit / unlexicalised
13	Customer care ne problem solve nahi ki, bas ghuma diya	Neg	Pos	Negation scope (domain noun)
14	Account se paise kat gaye aur order bhi nahi hua	Neg	Neu	Implicit / unlexicalised
15	Toss jeet ke bowling chuni	Neu	Pos	Domain-dependent polarity
16	Refund 5 se 7 din mein process hota hai	Neu	Neg	Domain-dependent polarity

Table 12. Targeted test sentences: gold label, output of the English-only baseline, output of the Hinglish engine and its compound score c (✓ correct, ✗ wrong).

Sentence	Gold	English only	Hinglish engine	c
movie acchi nahi thi	Neg	Neu ✗	Neg ✓	−0.361
bhot badiya	Pos	Neu ✗	Pos ✓	0.778
camera mast hai but battery bekaar	Neg	Neu ✗	Neg ✓	−0.612
match kal 7 baje hai	Neu	Neu ✓	Neu ✓	0.000
yaar ekdum bakwas film hai	Neg	Neu ✗	Neg ✓	−0.869
bilkul pasand nahi aaya	Neg	Neu ✗	Neg ✓	−0.502
service bahut slow hai lekin khana mast tha	Pos	Neg ✗	Pos ✓	0.599
help nahi ki 😡	Neg	Neg ✓	Neg ✓	−0.718
It isn’t bad	Pos	Pos ✓	Pos ✓	0.502
bakwas nahi thi, actually kaafi acchi nikli	Pos	Neu ✗	Pos ✓	0.822
not good at all	Neg	Neg ✓	Neg ✓	−0.361
movie bahut acchi thi 😍	Pos	Pos ✓	Pos ✓	0.881

Note: all figures were produced at 600 dpi with labelled axes, and the first row of every table is a header row.

Discussion:

A. Interpretation of the Findings

The central finding is that, when labelled data are scarce, accuracy on Hinglish is decided by vocabulary coverage rather than by the choice of algorithm. Adding Hinglish lexicon entries raised accuracy by 37 points over the same engine without them, whereas moving between Naive Bayes, logistic regression and a linear SVM changed accuracy by only a few points and never reliably. This agrees with the observation that simple linear models over n-grams are strong baselines whose quality depends mainly on the training data [24], and with reviews of under-resourced languages that recommend curated lexical resources [51]. For low-resource language pairs, a small, carefully curated lexicon appears to be an efficient first investment.

The hybrid improved clearly on Naive Bayes, by 16.2 points on average and by 25.6 points when only 17 training texts were available, but it did not beat the rule engine on this corpus. We do not take this as evidence that learning is useless. First, the rule engine was developed together with the corpus, so its advantage is inflated by construction. Second, the fusion rule is simple: Eq. (7) converts compound scores to probabilities in a way that suppresses weak rule signals, and Naive Bayes posteriors tend to be extreme, so a confident wrong learned vote can override a correct rule score, as in *Goa trip bahut mazedaar raha*. Calibrating both components and learning w by nested validation are natural refinements. Third, the real value of a learned component is to absorb domain vocabulary missing from the lexicon, and a corpus of 210 short, simple sentences cannot demonstrate this.

The ablations also teach a methodological lesson. Corpus-level accuracy did not react to three of the six rule components, yet removing any of them broke the targeted cases that had motivated them. Aggregate accuracy on a small and simple corpus can therefore exaggerate how complete a design is, and challenge sets built around specific phenomena such as post-negation, contrast,

sarcasm and code-switching points should accompany any benchmark [60]. The finding that post-negation corrected four texts and broke four shows that a linguistically motivated rule needs explicit exceptions, in this case for *kabhi nahi*, before it pays off.

B. Strengths and Practical Implications

Text Sentiment Analyser offers properties that large pre-trained models do not. Every verdict can be traced to lexicon entries and rules; the leave-one-out explainer shows the effect of function words; a text is analysed in about 0.1 ms; no network access is needed; and the REST interface is bound to the local machine. These qualities suit privacy-sensitive uses such as analysing student feedback or triaging customer-care messages on a local workstation, and they make the system a transparent baseline against which heavier models, including large language models [61, 63], can be compared. Because the lexicon is a plain data structure, a domain expert can extend it without any retraining.

C. Limitations and Threats to Validity

Several limitations qualify these conclusions. (1) The corpus is small, written by the project team, short (7.1 words on average) and artificially balanced, so it does not reproduce the noise, length and class imbalance of real social-media streams. (2) The lexicon and the corpus were developed together, so the rule-based accuracies are optimistic and cannot be compared with published benchmarks such as SAIL or SemEval-2020 Task 9 [10, 37]. (3) Inter-annotator agreement was not measured. (4) The hybrid weight is a fixed design constant, and the weight sweep uses the evaluation folds, so its optimum is an upper bound. (5) With 210 texts the bootstrap intervals span 7–13 points, and the ranking of the learned models changes with the split. (6) The emotion, aspect and sarcasm modules have no gold labels; they are described but not scored, and the emotion lexicon contains the *mein* defect identified above. (7) Only Roman-script text is analysed, sarcasm is handled by heuristics and aspects by keyword lists. (8) All timings come from a single environment. (9) The Python and JavaScript experiments use different folds and slightly different model settings, so their numbers are indicative rather than paired.

D. Future Work

The first priority is a proper evaluation: collecting at least 1,000 real comments from several domains, having two annotators label them and reporting kappa [72], freezing the lexicon, and testing on a set untouched during development. On such data, w and α should be chosen by nested cross-validation, the rule probabilities calibrated, and the defects found here repaired (multi-word emotion entries, *kabhi nahi* superlatives, more implicit-complaint words and variants such as *dheemi*); unsupervised normalisation [50] could replace part of the hand-built variant dictionary. Fine-tuning multilingual and Indic transformers, namely mBERT [79], XLM-R [40], MuRIL [33], IndicBERT [34] or HingBERT [32], would show whether the hybrid's advantage survives against much stronger learners. Further extensions include Devanagari input with transliteration, aspect-level evaluation, and sarcasm models trained on labelled data [56, 57].

Conclusion:

This paper presented Text Sentiment Analyser, a transparent sentiment and emotion analysis system for English and Hinglish that combines natural language processing, a rule-based

knowledge layer and machine learning in a privacy-preserving implementation with no external dependencies. On a balanced pilot corpus of 210 texts, an English-only lexicon reached 55.2% accuracy, the Hinglish-aware engine 92.4%, Naive Bayes and logistic regression about 66%, and the hybrid 82.1% on average over five splits, a significant 16-point improvement over Naive Bayes that grows to 26 points when training data are scarce. The ablations show that Romanised-Hindi vocabulary is the dominant factor, that corpus accuracy is insensitive to several rules that nevertheless correct targeted cases, and that Hindi post-negation needs exception handling for "never … such" idioms.

These results come with an important caveat: the rule engine was built alongside the corpus, so its score is an upper bound, and the data are too few and too simple to support claims about real social-media text. The contribution of the work lies in its transparent design, its reproducible evaluation protocol and its measured analysis of when each component helps. Evaluation on independent, doubly annotated data and comparison with pre-trained multilingual transformers are the next steps.

References:

1. Liu B.: Sentiment analysis and opinion mining. Synthesis Lectures on Human Language Technologies 5(1), 1–167 (2012). 10.2200/S00416ED1V01Y201204HLT016
2. Pang B., Lee L.: Opinion mining and sentiment analysis. Foundations and Trends in Information Retrieval 2(1–2), 1–135 (2008). 10.1561/1500000011
3. Birjali M., Kasri M., Beni-Hssane A.: A comprehensive survey on sentiment analysis: Approaches, challenges and trends. Knowledge-Based Systems 226, 107134 (2021). 10.1016/j.knosys.2021.107134
4. Hutto C.J., Gilbert E.: VADER: A parsimonious rule-based model for sentiment analysis of social media text. Proceedings of the International AAAI Conference on Web and Social Media 8(1), 216–225 (2014). 10.1609/icwsm.v8i1.14550
5. Taboada M., Brooke J., Tofiloski M., et al.: Lexicon-based methods for sentiment analysis. Computational Linguistics 37(2), 267–307 (2011). 10.1162/COLI_a_00049
6. Pang B., Lee L., Vaithyanathan S.: Thumbs up? Sentiment classification using machine learning techniques. In: Proceedings of the 2002 Conference on Empirical Methods in Natural Language Processing, pp. 79–86. Association for Computational Linguistics (2002). 10.3115/1118693.1118704
7. Bali K., Sharma J., Choudhury M., et al.: "I am borrowing ya mixing?" An analysis of English-Hindi code mixing in Facebook. In: Proceedings of the First Workshop on Computational Approaches to Code Switching, pp. 116–126. Association for Computational Linguistics (2014). 10.3115/v1/W14-3914
8. Barman U., Das A., Wagner J., et al.: Code mixing: A challenge for language identification in the language of social media. In: Proceedings of the First Workshop on Computational Approaches to Code Switching, pp. 13–23. Association for Computational Linguistics (2014). 10.3115/v1/W14-3902
9. Vyas Y., Gella S., Sharma J., et al.: POS tagging of English-Hindi code-mixed social media content. In: Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, pp. 974–979. Association for Computational Linguistics (2014). 10.3115/v1/D14-1105
10. Patra B.G., Das D., Das A.: Sentiment analysis of code-mixed Indian languages: An overview of SAIL_Code-Mixed shared task @ICON-2017. arXiv:1803.06745 (2018). 10.48550/arXiv.1803.06745
11. Wankhade M., Rao A.C.S., Kulkarni C.: A survey on sentiment analysis methods, applications, and challenges. Artificial Intelligence Review 55(7), 5731–5780 (2022). 10.1007/s10462-022-10144-1

12. Nandwani P., Verma R.: A review on sentiment analysis and emotion detection from text. Social Network Analysis and Mining 11(1), 81 (2021). 10.1007/s13278-021-00776-6
13. Bordoloi M., Biswas S.K.: Sentiment analysis: A survey on design framework, applications and future scopes. Artificial Intelligence Review 56(11), 12505–12560 (2023). 10.1007/s10462-023-10442-2
14. Tan K.L., Lee C.P., Lim K.M.: A survey of sentiment analysis: Approaches, datasets, and future research. Applied Sciences 13(7), 4550 (2023). 10.3390/app13074550
15. Rodríguez-Ibánez M., Casánez-Ventura A., Castejón-Mateos F., et al.: A review on sentiment analysis from social media platforms. Expert Systems with Applications 223, 119862 (2023). 10.1016/j.eswa.2023.119862
16. Jim J.R., Talukder M.A.R., Malakar P., et al.: Recent advancements and challenges of NLP-based sentiment analysis: A state-of-the-art review. Natural Language Processing Journal 6, 100059 (2024). 10.1016/j.nlp.2024.100059
17. Jain P.K., Pamula R., Srivastava G.: A systematic literature review on machine learning applications for consumer sentiment analysis using online reviews. Computer Science Review 41, 100413 (2021). 10.1016/j.cosrev.2021.100413
18. Hartmann J., Heitmann M., Siebert C., et al.: More than a feeling: Accuracy and application of sentiment analysis. International Journal of Research in Marketing 40(1), 75–87 (2023). 10.1016/j.ijresmar.2022.05.005
19. Cambria E., Liu Q., Decherchi S., et al.: SenticNet 7: A commonsense-based neurosymbolic AI framework for explainable sentiment analysis. In: Proceedings of the 13th Language Resources and Evaluation Conference, pp. 3829–3839. European Language Resources Association (2022).
20. Catelli R., Pelosi S., Esposito M.: Lexicon-based vs. BERT-based sentiment analysis: A comparative study in Italian. Electronics 11(3), 374 (2022). 10.3390/electronics11030374
21. Appel O., Chiclana F., Carter J., et al.: A hybrid approach to the sentiment analysis problem at the sentence level. Knowledge-Based Systems 108, 110–124 (2016). 10.1016/j.knosys.2016.05.040
22. Kaur G., Sharma A.: A deep learning-based model using hybrid feature extraction approach for consumer sentiment analysis. Journal of Big Data 10, Article 5 (2023). 10.1186/s40537-022-00680-6
23. Alsayat A.: Improving sentiment analysis for social media applications using an ensemble deep learning language model. Arabian Journal for Science and Engineering 47(2), 2499–2511 (2022). 10.1007/s13369-021-06227-w
24. Wang S., Manning C.D.: Baselines and bigrams: Simple, good sentiment and topic classification. In: Proceedings of the 50th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers), pp. 90–94. Association for Computational Linguistics (2012).
25. Devlin J., Chang M.-W., Lee K., et al.: BERT: Pre-training of deep bidirectional transformers for language understanding. In: Proceedings of NAACL-HLT 2019, pp. 4171–4186. Association for Computational Linguistics (2019). 10.18653/v1/N19-1423
26. Tan K.L., Lee C.P., Anbananthen K.S.M., et al.: RoBERTa-LSTM: A hybrid model for sentiment analysis with transformer and recurrent neural network. IEEE Access 10, 21517–21525 (2022). 10.1109/ACCESS.2022.3152828
27. Barbieri F., Espinosa Anke L., Camacho-Collados J.: XLM-T: Multilingual language models in Twitter for sentiment analysis and beyond. In: Proceedings of the 13th Language Resources and Evaluation Conference, pp. 258–266. European Language Resources Association (2022).
28. Doğruöz A.S., Sitaram S., Bullock B.E., et al.: A survey of code-switching: Linguistic and social perspectives for language technologies. In: Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing, pp. 1654–1666 (2021). 10.18653/v1/2021.acl-long.131
29. Winata G.I., Aji A.F., Yong Z.-X., et al.: The decades progress on code-switching research in NLP: A systematic survey on trends and challenges. In: Findings of the Association for Computational Linguistics: ACL 2023, pp. 2936–2978 (2023). 10.18653/v1/2023.findings-acl.185

30. Aguilar G., Kar S., Solorio T.: LinCE: A centralized benchmark for linguistic code-switching evaluation. In: Proceedings of the 12th Language Resources and Evaluation Conference, pp. 1803–1813. European Language Resources Association (2020).
31. Khanuja S., Dandapat S., Srinivasan A., et al.: GLUECoS: An evaluation benchmark for code-switched NLP. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pp. 3575–3585 (2020). 10.18653/v1/2020.acl-main.329
32. Nayak R., Joshi R.: L3Cube-HingCorpus and HingBERT: A code mixed Hindi-English dataset and BERT language models. In: Proceedings of the WILDRE-6 Workshop within the 13th Language Resources and Evaluation Conference, pp. 7–12. European Language Resources Association (2022).
33. Khanuja S., Bansal D., Mehtani S., et al.: MuRIL: Multilingual representations for Indian languages. arXiv:2103.10730 (2021). 10.48550/arXiv.2103.10730
34. Kakwani D., Kunchukuttan A., Golla S., et al.: IndicNLPSuite: Monolingual corpora, evaluation benchmarks and pre-trained multilingual language models for Indian languages. In: Findings of the Association for Computational Linguistics: EMNLP 2020, pp. 4948–4961 (2020). 10.18653/v1/2020.findings-emnlp.445
35. Joshi A., Prabhu A., Shrivastava M., et al.: Towards sub-word level compositions for sentiment analysis of Hindi-English code mixed text. In: Proceedings of COLING 2016, the 26th International Conference on Computational Linguistics: Technical Papers, pp. 2482–2491. The COLING 2016 Organizing Committee (2016).
36. Sarkar K.: JU_KS@SAIL_CodeMixed-2017: Sentiment analysis for Indian code mixed social media texts. arXiv:1802.05737 (2018). 10.48550/arXiv.1802.05737
37. Patwa P., Aguilar G., Kar S., et al.: SemEval-2020 Task 9: Overview of sentiment analysis of code-mixed tweets. In: Proceedings of the Fourteenth Workshop on Semantic Evaluation, pp. 774–790. International Committee for Computational Linguistics (2020). 10.18653/v1/2020.semeval-1.100
38. Srinivasan A.: MSR India at SemEval-2020 Task 9: Multilingual models can do code-mixing too. In: Proceedings of the Fourteenth Workshop on Semantic Evaluation, pp. 951–956. International Committee for Computational Linguistics (2020).
39. Ma Y., Zhao L., Hao J.: XLP at SemEval-2020 Task 9: Cross-lingual models with focal loss for sentiment analysis of code-mixing language. In: Proceedings of the Fourteenth Workshop on Semantic Evaluation, pp. 975–980. International Committee for Computational Linguistics (2020).
40. Conneau A., Khandelwal K., Goyal N., et al.: Unsupervised cross-lingual representation learning at scale. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pp. 8440–8451. Association for Computational Linguistics (2020). 10.18653/v1/2020.acl-main.747
41. Kumar A., Agarwal H., Bansal K., et al.: BAKSA at SemEval-2020 Task 9: Bolstering CNN with self-attention for sentiment analysis of code mixed text. In: Proceedings of the Fourteenth Workshop on Semantic Evaluation, pp. 1221–1226. International Committee for Computational Linguistics (2020).
42. Singh P., Lefever E.: Sentiment analysis for Hinglish code-mixed tweets by means of cross-lingual word embeddings. In: Proceedings of the 4th Workshop on Computational Approaches to Code Switching, pp. 45–51. European Language Resources Association (2020).
43. Baroi S.J., Singh N., Das R., et al.: NITS-Hinglish-SentiMix at SemEval-2020 Task 9: Sentiment analysis for code-mixed social media text using an ensemble model. In: Proceedings of the Fourteenth Workshop on Semantic Evaluation. International Committee for Computational Linguistics (2020). 10.48550/arXiv.2007.12081
44. Chakravarthi B.R., Muralidaran V., Priyadharshini R., et al.: Corpus creation for sentiment analysis in code-mixed Tamil-English text. In: Proceedings of the 1st Joint Workshop on Spoken Language Technologies for Under-resourced Languages (SLTU) and Collaboration and Computing for Under-Resourced Languages (CCURL), pp. 202–210. European Language Resources Association (2020).
45. Chakravarthi B.R., Jose N., Suryawanshi S., et al.: A sentiment analysis dataset for code-mixed Malayalam-English. In: Proceedings of the 1st Joint Workshop on Spoken Language Technologies

for Under-resourced Languages (SLTU) and Collaboration and Computing for Under-Resourced Languages (CCURL), pp. 177–184. European Language Resources Association (2020).

46. Chakravarthi B.R., Priyadharshini R., Muralidaran V., et al.: DravidianCodeMix: Sentiment analysis and offensive language identification dataset for Dravidian languages in code-mixed text. Language Resources and Evaluation 56(3), 765–806 (2022). 10.1007/s10579-022-09583-7
47. Hande A., Priyadharshini R., Chakravarthi B.R.: KanCMD: Kannada CodeMixed dataset for sentiment analysis and offensive language detection. In: Proceedings of the Third Workshop on Computational Modeling of People's Opinions, Personality, and Emotion's in Social Media, pp. 54–63. Association for Computational Linguistics (2020).
48. Bohra A., Vijay D., Singh V., et al.: A dataset of Hindi-English code-mixed social media text for hate speech detection. In: Proceedings of the Second Workshop on Computational Modeling of People's Opinions, Personality, and Emotions in Social Media, pp. 36–41. Association for Computational Linguistics (2018). 10.18653/v1/W18-1105
49. Rani P., Suryawanshi S., Goswami K., et al.: A comparative study of different state-of-the-art hate speech detection methods in Hindi-English code-mixed data. In: Proceedings of the Second Workshop on Trolling, Aggression and Cyberbullying, pp. 42–48. European Language Resources Association (2020).
50. Mehmood K., Essam D., Shafi K., et al.: An unsupervised lexical normalization for Roman Hindi and Urdu sentiment analysis. Information Processing & Management 57(6), 102368 (2020). 10.1016/j.ipm.2020.102368
51. Mabokela K.R., Celik T., Raborife M.: Multilingual sentiment analysis for under-resourced languages: A systematic review of the landscape. IEEE Access 11, 15996–16020 (2023). 10.1109/ACCESS.2022.3224136
52. Plutchik R.: A general psychoevolutionary theory of emotion. In: Plutchik R., Kellerman H. (eds.) Emotion: Theory, Research, and Experience, vol. 1, pp. 3–33. Academic Press, New York (1980).
53. Mohammad S.M., Turney P.D.: Crowdsourcing a word–emotion association lexicon. Computational Intelligence 29(3), 436–465 (2013). 10.1111/j.1467-8640.2012.00460.x
54. Demszky D., Movshovitz-Attias D., Ko J., et al.: GoEmotions: A dataset of fine-grained emotions. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pp. 4040–4054 (2020). 10.18653/v1/2020.acl-main.372
55. Acheampong F.A., Wenyu C., Nunoo-Mensah H.: Text-based emotion detection: Advances, challenges, and opportunities. Engineering Reports 2(7), e12189 (2020). 10.1002/eng2.12189
56. González-Ibáñez R., Muresan S., Wacholder N.: Identifying sarcasm in Twitter: A closer look. In: Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies, pp. 581–586. Association for Computational Linguistics (2011).
57. Kumar A., Narapareddy V.T., Srikanth V.A., et al.: Sarcasm detection using multi-head attention based bidirectional LSTM. IEEE Access 8, 6388–6397 (2020). 10.1109/ACCESS.2019.2963630
58. Ribeiro M.T., Singh S., Guestrin C.: "Why should I trust you?" Explaining the predictions of any classifier. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 1135–1144. ACM (2016). 10.1145/2939672.2939778
59. Atanasova P., Simonsen J.G., Lioma C., et al.: A diagnostic study of explainability techniques for text classification. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, pp. 3256–3274 (2020). 10.18653/v1/2020.emnlp-main.263
60. Ribeiro M.T., Wu T., Guestrin C., et al.: Beyond accuracy: Behavioral testing of NLP models with CheckList. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pp. 4902–4912 (2020). 10.18653/v1/2020.acl-main.442
61. Zhang W., Deng Y., Liu B., et al.: Sentiment analysis in the era of large language models: A reality check. In: Findings of the Association for Computational Linguistics: NAACL 2024, pp. 3881–3906. Association for Computational Linguistics (2024). 10.18653/v1/2024.findings-naacl.246

62. Zhang W., Li X., Deng Y., et al.: A survey on aspect-based sentiment analysis: Tasks, methods, and challenges. IEEE Transactions on Knowledge and Data Engineering 35(11), 11019–11038 (2023). 10.1109/TKDE.2022.3230975
63. Zhang R., Cahyawijaya S., Cruz J.C.B., et al.: Multilingual large language models are not (yet) code-switchers. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, pp. 12567–12582 (2023). 10.18653/v1/2023.emnlp-main.774
64. Pedregosa F., Varoquaux G., Gramfort A., et al.: Scikit-learn: Machine learning in Python. Journal of Machine Learning Research 12, 2825–2830 (2011).
65. Kralj Novak P., Smailović J., Sluban B., et al.: Sentiment of emojis. PLoS ONE 10(12), e0144296 (2015). 10.1371/journal.pone.0144296
66. McCallum A., Nigam K.: A comparison of event models for naive Bayes text classification. In: AAAI-98 Workshop on Learning for Text Categorization, pp. 41–48. AAAI Press (1998).
67. Salton G., Buckley C.: Term-weighting approaches in automatic text retrieval. Information Processing & Management 24(5), 513–523 (1988). 10.1016/0306-4573(88)90021-0
68. Manning C.D., Raghavan P., Schütze H.: Introduction to information retrieval. Cambridge University Press (2008). 10.1017/CBO9780511809071
69. Bottou L.: Large-scale machine learning with stochastic gradient descent. In: Proceedings of COMPSTAT’2010, pp. 177–186. Physica-Verlag (2010). 10.1007/978-3-7908-2604-3_16
70. Cortes C., Vapnik V.: Support-vector networks. Machine Learning 20(3), 273–297 (1995). 10.1007/BF00994018
71. Kohavi R.: A study of cross-validation and bootstrap for accuracy estimation and model selection. In: Proceedings of the 14th International Joint Conference on Artificial Intelligence, pp. 1137–1143. Morgan Kaufmann (1995).
72. Cohen J.: A coefficient of agreement for nominal scales. Educational and Psychological Measurement 20(1), 37–46 (1960). 10.1177/001316446002000104
73. Efron B., Tibshirani R.J.: An introduction to the bootstrap. Chapman & Hall (1993).
74. McNemar Q.: Note on the sampling error of the difference between correlated proportions or percentages. Psychometrika 12(2), 153–157 (1947). 10.1007/BF02295996
75. Dietterich T.G.: Approximate statistical tests for comparing supervised classification learning algorithms. Neural Computation 10(7), 1895–1923 (1998). 10.1162/089976698300017197
76. Landis J.R., Koch G.G.: The measurement of observer agreement for categorical data. Biometrics 33(1), 159–174 (1977). 10.2307/2529310
77. Harris C.R., Millman K.J., van der Walt S.J., et al.: Array programming with NumPy. Nature 585(7825), 357–362 (2020). 10.1038/s41586-020-2649-2
78. Hunter J.D.: Matplotlib: A 2D graphics environment. Computing in Science & Engineering 9(3), 90–95 (2007). 10.1109/MCSE.2007.55
79. Pires T., Schlinger E., Garrette D.: How multilingual is multilingual BERT? In: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, pp. 4996–5001. Association for Computational Linguistics (2019). 10.18653/v1/P19-1493

Conflict of Interest Statement:

The authors have no conflict of interest to disclose.

Acknowledgments:

The authors thank the Department of Computer Science and Engineering, Maharana Institute of Professional Studies, Kanpur, for its support and guidance, and acknowledge the open-source communities behind Node.js, Python, scikit-learn, NumPy and Matplotlib.